

Seralix: A Minimal Language Prototype for Expressing Fast Fourier Transform Algorithms

David Mateo Barbosa Monsalve
Universidad de los Andes
Ingeniería de Sistemas y Computación

Abstract—This paper presents Seralix, a minimal programming language prototype designed to explore language construction through the implementation of the Fast Fourier Transform (FFT). The project demonstrates how a custom language can express complex algorithms while maintaining syntactic simplicity. The correctness of the language is validated through recursive FFT and polynomial multiplication.

I. INTRODUCTION

The Fast Fourier Transform (FFT) is a fundamental algorithm used in signal processing, numerical methods, and computational algebra. Its efficiency makes it essential for polynomial multiplication and large-scale computations.

This project explores the design of a programming language, Seralix, as a way to express such algorithms while understanding the underlying computational structure.

II. FUNDAMENTOS TEÓRICOS

A. Transformada Discreta de Fourier

La DFT de una secuencia $a_0, \dots, a_{n-1}$ se define como:

$$A_k = \sum_{j=0}^{n-1} a_j \omega_n^{jk} \quad (1)$$

donde $\omega_n = e^{2\pi i/n}$.

Este cálculo requiere n sumas de tamaño n , resultando en:

$$O(n^2) \quad (2)$$

B. FFT y Descomposición

La FFT utiliza una descomposición en términos pares e impares:

$$A_k = A_k^{(even)} + \omega_n^k A_k^{(odd)} \quad (3)$$

reduciendo el problema a dos subproblemas de tamaño $n/2$.

C. Análisis de Complejidad

La recurrencia es:

$$T(n) = 2T(n/2) + O(n) \quad (4)$$

Aplicando el Teorema Maestro:

$$T(n) = O(n \log n) \quad (5)$$

Esto ocurre porque el algoritmo realiza $\log n$ niveles de recursión, cada uno con costo lineal.

D. Sobre el Límite Inferior

No existe una prueba universal de que $O(n \log n)$ sea un límite absoluto, pero resultados teóricos indican cotas inferiores de tipo:

$$\Omega(n \log n) \quad (6)$$

bajo modelos restringidos de computación, lo que sugiere que la FFT es esencialmente óptima.

E. Multiplicación de Polinomios

La multiplicación directa tiene costo:

$$O(n^2) \quad (7)$$

pero usando FFT se reduce a:

$$O(n \log n) \quad (8)$$

mediante evaluación, multiplicación punto a punto y transformada inversa.

III. MOTIVACIÓN DEL LENGUAJE

El objetivo de Seralix no es reemplazar lenguajes existentes, sino explorar cómo el diseño de un lenguaje influye en la expresión de algoritmos.

FFT requiere:

- Recursión
- Manejo de secuencias
- Operaciones complejas

Seralix permite exponer explícitamente estos elementos, en lugar de abstraerlos completamente como en lenguajes tradicionales.

IV. ARQUITECTURA DEL SISTEMA

El flujo del lenguaje es:

Código $\rightarrow$ Lexer $\rightarrow$ Parser $\rightarrow$ AST $\rightarrow$ Intérprete

V. DISEÑO DEL LENGUAJE

A. Funciones

```
(x) -> x * x
```

```
function add(x, y=2) {  
    return x + y;  
}
```

B. Control de Flujo

```
if x < 0 {  
    ...  
} else {  
    ...  
}
```

C. Loops

```
while i < 10 {  
    i += 1;  
}
```

VI. IMPLEMENTACIÓN FFT

```
function fft(a, invert=false) {  
    if len(a) == 1 {  
        return [a[0]];  
    }  
  
    even = fft(a[0::2], invert);  
    odd = fft(a[1::2], invert);  
}
```

VII. RESULTADOS

```
a = [1,2,3];  
b = [4,5,6];
```

Resultado:

```
[4,13,28,27,18]
```

VIII. DISCUSIÓN

El lenguaje permite:

- Implementar recursión
- Ejecutar FFT correctamente
- Expresar algoritmos complejos

Limitaciones:

- Falta de optimización
- Errores de precisión flotante

IX. CONCLUSIÓN

Se demuestra que es posible diseñar un lenguaje capaz de ejecutar algoritmos de complejidad óptima como FFT, validando la capacidad del autor para construir sistemas de lenguaje desde cero.

X. REFERENCIAS

REFERENCES

- [1] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," 1965.
- [2] "Fast Fourier Transform," Wikipedia.
- [3] P. Duhamel and M. Vetterli, "Fast Fourier transforms: a tutorial review," 1990.
- [4] N. Ailon, "Lower bounds for Fourier Transform computation," 2013.